



Генерация приложений анализа сетевого трафика с помощью LLM

Команда: **Гении промтинга** 🧐

Студенты группы 130 (ПОВС)

Рябыкин В.А., Опрышко М.И, Письменная Е.С., Пионткевич А.Г.

Москва, 2026

Постановка задачи

Вопрос: можно ли создать Python-приложение для классификации вредоносного / атакующего трафика только промптом к LLM?

Задачи:

- Сформировать промпты (EN и RU)
- Сгенерировать код с использованием 3 LLM (GPT-5.5, Opus 4.6, YandexAliceAI)
- Сравнить с моделью, написанной вручную
- Опубликовать репозиторий
- Подвести итоги по проделанной работе

Сведения о датасете

Параметр	Значение
Источник	Stratosphere Laboratory. A labeled dataset with malicious and benign IoT network traffic (первый файл)
Объект	Запись о сетевом соединении (23 признака)
Число объектов	1008748
Цель	label: 1 (Benign) — норма, 0 (Malicious) — атака

Прототип: бинарная классификация

Промпт (русский)

Файл: *prompts/prompt_ru.md*

Первая версия написана с использованием Composer 2.0. Далее - правки руками

Ключевые требования к LLM, описанные в промте:

- 1) Загрузка CSV из *data/raw*
- 2) Исключить избыточные признаки: *id* и поля с *IP* (*srcip*, *dstip* и т. д)
- 3) Корректно обработать данные датасета (заполнить пропуски, кодировать категориальные признаки, масштабирование числовых признаков)
- 4) Модели предложен заголовок CSV и первая строка
- 5) Разбиение train/test на усмотрение LLM, *random_state* константа
- 6) Выбор модели также на усмотрение LLM
- 7) Вывести метрики: *accuracy*, *precision*, *recall*, *F1*, *матрицу ошибок*.
- 8) Использовать только: *pandas*, *numpy*, *scikit-learn*, *joblib*
- 9) Сохраните результаты промпта в единый файл *malware_classifier.py*

Промпт (английский)

Файл: *prompts/prompt_en.md*

Аналогичный запрос, но на английском языке

Гипотеза: язык промпта не должен снижать качество у сильных мультязычных моделей.

Сравнение LLM

Модель	Промпт	Accuracy	Precision	Recall	F1	#LoC
GPT-5.5	EN	0.956198	0.925025	0.999073	0.960624	515
GPT-5.5	RU	0.956203	0.925018	0.999092	0.960629	581
Claude 4.6 Opus	EN	0.956565	0.924970	0.999889	0.960972	319
Claude 4.6 Opus	RU	0.956570	0.924978	0.999889	0.960976	302
Yandex Alice AI	EN	0.956213	0.925012	0.999120	0.960639	171
Yandex Alice AI	RU	0.956213	0.925012	0.999120	0.960639	171
Hand-coded baseline	MSU	0.956550	0.924990	0.999833	0.960957	156

Использованные модели

GPT-5.5o; EN: RandomForestClassifier(n_estimators=200, max_depth=None)

GPT-5.5o; RU: RandomForestClassifier (n_estimators=200, max_depth=None)

Claude 4.6 Opus; EN: RandomForestClassifier (n_estimators=300, max_depth=6)

Claude 4.6 Opus; RU: GradientBoostingClassifier (n_estimators=200, max_depth=25)

Yandex Alice AI; EN: RandomForestClassifier(n_estimators=100)

Yandex Alice AI; RU: RandomForestClassifier(n_estimators=100)

Ручная модель: RandomForestClassifier(n_estimators=300) или

GradientBoostingClassifier(n_estimators=150, max_depth=5)

1. Рабочее IDS-приложение на sklearn реально получить из промпта
2. GPT-5.5o и Opus при одном EN-промpte дают сопоставимый код.
3. Русский промпт не ухудшил результат (при мультязычной модели).
4. Русский промпт приводит к тому, что LLM пишет комментарии в коде на русском языке, даже без явного ей на то указания

GitHub: *https://github.com/VladislavRya/CMC_iot_prompt_project*

Запуск:

pip install -r requirements.txt

bash scripts/download_kaggle_data.sh

python scripts/run_comparison.py

Спасибо за внимание!

С вами были **Гении промтинга** 😎